

# Making Neural Networks Work

## UW ECE 657A - Core Topic

Mark Crowley

# Lecture Outline

- 1 Problems with Neural Networks
- 2 Overfitting
- 3 Vanishing Gradients
- 4 Catastrophic Forgetting
- 5 Interpretability

# Problems with Neural Networks

- Overfitting
- Very inefficient for images, timeseries, large numbers of inputs-outputs
- Slow to train
- Overfitting
- Vanishing Gradients
- Catastrophic Forgetting
- Interpretability is Very Hard

# Heuristics for Improving Backpropagation

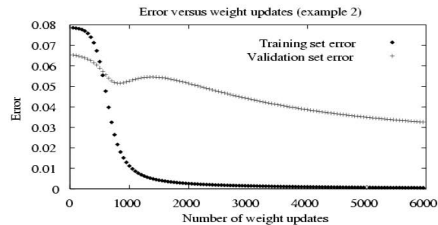
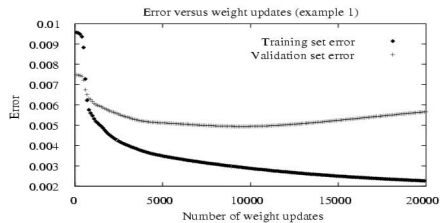
There are a number of useful heuristics for training Neural Networks that are useful in practice (maybe we'll learn more today):

- Less hidden nodes, just enough complexity to work, not too much to overfit.
- Train multiple networks with different sizes and search for the best design.
- Validation set: train on training set until error on validation set starts to rise, then evaluate on evaluation set.
- Try different activation functions: tanh, ReLU, ELU, ...?
- Dropout (Hinton 2014) - randomly ignore certain units during training, don't update them via gradient descent, leads to hidden units that specialize
- Modify learning rate over time (cooling schedule)

# Lecture Outline

- 1 Problems with Neural Networks
- 2 Overfitting
- 3 Vanishing Gradients
- 4 Catastrophic Forgetting
- 5 Interpretability

# The Threat of Overfitting



# Problem: Overfitting

Solution: Standard Regularization

- L2 regularization - augmenting the error function, add  $\frac{1}{2}\lambda w^2$  to all weights in the neural network
  - Need to choose *lambda* carefully
  - Interpretation - heavily penalizing "peaky" weight vectors and preferring diffuse weight vectors.
  - Encourages network to use all its inputs instead of relying on strongest signal.
  - Also called **weight decay** since weights would reduce over time during gradient descent without input.
- L1 regularization - add  $\lambda|w|$  to each weight error function (encourages all weights to zero)

# Problem: Overfitting

Solution: Standard Regularization

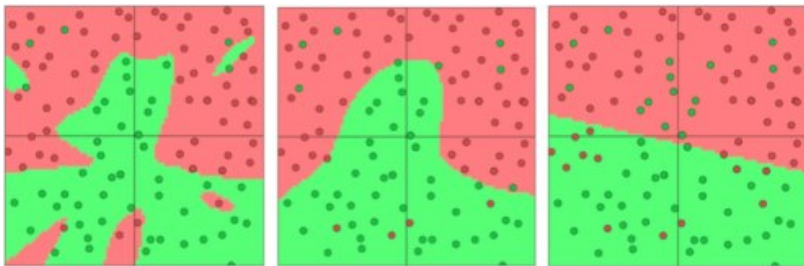


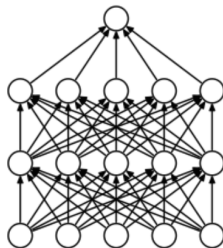
Figure: From <http://www.kdnuggets.com/2015/04/preventing-overfitting-neural-networks.html/2>



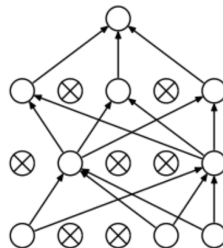
# Problem: Overfitting

## Solution: Dropout

- Dropout (Hinton 2014) - randomly ignore certain units during training, don't update them via gradient descent, leads to hidden units that specialize.
- With probability  $p$  don't include a weight in the gradient updates.
- Reduces overfitting by encouraging robustness of weights in the network.



Standard Neural Net



After applying dropout.

# Lecture Outline

- 1 Problems with Neural Networks
- 2 Overfitting
- 3 Vanishing Gradients**
- 4 Catastrophic Forgetting
- 5 Interpretability

# Problem: Vanishing Gradient

- This was not an issue in the original Neural Networks with just a few hidden layers
- As more hidden layers are added, the backpropagation algorithm is multiplying more small gradients over and over.

# Problem: Vanishing Gradient

- This was not an issue in the original Neural Networks with just a few hidden layers
- As more hidden layers are added, the backpropagation algorithm is multiplying more small gradients over and over.
- so what happens?....

# Problem: Vanishing Gradient

- This was not an issue in the original Neural Networks with just a few hidden layers
- As more hidden layers are added, the backpropagation algorithm is multiplying more small gradients over and over.
- so what happens?....
- The gradients converge to zero and become meaningless.

# Problem: Vanishing Gradient

- This was not an issue in the original Neural Networks with just a few hidden layers
- As more hidden layers are added, the backpropagation algorithm is multiplying more small gradients over and over.
- so what happens?....
- The gradients converge to zero and become meaningless.
- They can also diverge to infinity, which is just as bad, this is the **exploding gradient** problem

# Problem: Vanishing Gradient

- This was not an issue in the original Neural Networks with just a few hidden layers
- As more hidden layers are added, the backpropagation algorithm is multiplying more small gradients over and over.
- so what happens?....
- The gradients converge to zero and become meaningless.
- They can also diverge to infinity, which is just as bad, this is the **exploding gradient** problem

So how can we fix this problem?

# Problem: Vanishing Gradient

Solution: More backward linking

- Hinton has said he wished we could find a better algorithm than Backpropagation because of this and other problems.
- ReLu helps. **Why?**
- Links back more than one layer. **Why would this help?**



# Inception and Resnet

See Inception/Resnet slide deck

# Lecture Outline

- 1 Problems with Neural Networks
- 2 Overfitting
- 3 Vanishing Gradients
- 4 Catastrophic Forgetting**
- 5 Interpretability

# Problem: Catastrophic Forgetting

Solution: importance weighting of parameters

- What if once trained on a given dataset, you provide the network *new data* to train on?
- What if that data comes from a somewhat different, or *very different* distribution?
- Will the network:
  - ① smoothly integrate the new knowledge while not degrading performance on the original data?
  - ② “overwrite” the original distributions and weights given the new data?
  - ③ raise an alert that the new data comes from a different distribution with confidence level  $\sigma\%$ ?

# Problem: Catastrophic Forgetting

Solution: importance weighting of paramters

This is only a concern if your data has **distribution shifting** or if you are performing **Tranfser Learning** or want **Continual Learning**.

# Lecture Outline

- 1 Problems with Neural Networks
- 2 Overfitting
- 3 Vanishing Gradients
- 4 Catastrophic Forgetting
- 5 Interpretability

# Problem: Unintertpretable Neural Network Model

- This is a larger problem but interesting to highlight here.
- What do we *mean* that a model is not **interpretable**?
- What problems could arise from not being able to interpret your model?
- See Interpretability slides.

# Problem: Unintertpretable Neural Network Model

Solution: Attention Mechanisms

**Attention:** to focus on some area of the world with more intensity.

- In Deep Learning, **attention** means a specific approach to updating your network weights

# Problems and Solutions

## 1 Problems with Neural Networks

## 2 Overfitting

- Regularization
- Dropout

## 3 Vanishing Gradients

- Backward Linking

## 4 Catastrophic Forgetting

- Importance Weighting

## 5 Interpretability

- Attention